

Sliding Window

Java cheat sheet - fixed size and variable size templates

1. Sliding window - fixed size (size k)

When: window size k is given up front (max/min sum or average of any k consecutive elements).

```
public int maxSumFixedWindow(int[] arr, int k) {
    int n = arr.length;
    int windowSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }
    int maxSum = windowSum;

    for (int right = k; right < n; right++) {
        windowSum += arr[right];          // naya element add
        windowSum -= arr[right - k];      // sabse purana element hatao
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

Key idea: the window slides, it never shrinks or grows. $\text{left} = \text{right} - k + 1$, so you don't even need a left variable.

Examples: Max Sum Subarray of Size K, Average of Subarrays of Size K.

2. Sliding window - variable size (expand + shrink)

When: window size is not fixed - find smallest/longest window meeting a condition (min subarray with sum $\geq$ target, longest substring without repeating chars).

```
public int minSubArrayLen(int target, int[] nums) {
    int left = 0, sum = 0;
    int minLen = Integer.MAX_VALUE;

    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];          // expand - hamesha

        while (sum >= target) {      // shrink - jab tak zaroorat
            minLen = Math.min(minLen, right - left + 1);
            sum -= nums[left];
            left++;
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}
```

Key idea: right moves every iteration (n times total); left only moves when needed. Together at most $2n$ pointer moves $\rightarrow O(n)$, even though it looks like a nested loop.

Examples: Minimum Size Subarray Sum, Longest Substring Without Repeating Characters, Longest Substring with At Most K Distinct Characters, Fruit Into Baskets.

Fixed vs variable - quick comparison

	Fixed window	Variable window
--	--------------	-----------------

left pointer	implicit: $\text{right} - k + 1$	explicit, moves only when condition needs it
right pointer	moves every step	moves every step
loop shape	single for loop	for loop with inner while
when to use	window size k is given	looking for smallest/longest window